

IMPERIAL

Building and Running Scientific Software

From a Research Infrastructure Engineer's Perspective

Durham HPC Days 2026

Dr Dugan Witherick (Head of Research Computing Platforms)

The aim of this talk (and why this topic matters)

- Researchers and Research Software Engineers (users) want to run their scientific applications on HPC/accelerated compute resources.
- Research Infrastructure Engineers (RIEs) sit between the users and compute facilities and often have a good understanding of what does and doesn't work well on those systems.
- This talk will hopefully provide you useful insights to get your program up and running sooner on a compute facility.



Topics covered

- Consider the “full stack”
- Code Compilation (with gcc)
- Scientific software build tools
- Containers

The advice presented here is trying to be broad, and is based on the experience of the speaker, supported by colleagues in the ACIT hub.

If the advice provided contradicts your local RIE teams, always follow the advice of your local team as they know their system better than we do!

Consider the entire stack



Operating system

RedHat Enterprise Linux (RHEL) or variant (often), Ubuntu (increasingly), SuSE (Cray?)

Scheduler

Affects process placement

Slurm, PBS Pro, Grid Engine (variants)

Hardware

CPU micro-architecture

CPU interconnect

NUMA domains

Network or Interconnect

Ethernet

RoCE (RDMA over Converged Ethernet)

Infiniband

HPE Slingshot

File system

GPFS
Spectrum Scale

Weka

Lustre

VAST

Why do so many HPC systems still use RHEL (or variants)

Typically chosen for practical, long term engineering reasons rather than ideology or preference.

Benefits of RHEL:

- Slow, conservative release cycle with long support lifetimes.
- Vendor/hardware support for specialist devices
- Device drivers are often certified and optimised first for RHEL.
 - GPUs,
 - high speed interconnects (Infiniband/RoCE),
 - storage drivers (GPFS, Weka, Lustre).
- Easier to test/compare scientific software against other sites running the same OS.

Updating/patching an HPC system can be complicated:

- Lots of servers to update.
- Patching can affect research software so needs to be tested first.
- Specialist/highly tuned device drivers need to be compatible with OS updates/patches.

Why we might ask you to “use a newer version” of GCC

Older programs may fail to compile on newer GCC versions:

- Modern compilers are stricter
- Follow updated language standards more closely
- Remove support for outdated or unsafe features

Older code may have been reliant on:

- Non-standard extensions
- Deprecated syntax
- Outdated libraries or build tools

Code reliant on old version of GCC

- GCC may not know about or understand your CPU
- Could fall back to failsafe option, resulting in either very slow software or even completely wrong output

Every version of linux is based on a gcc/glibc version and it can be very difficult to get that older version of gcc installed:

- Older gcc versions can be dependent on
 - Old versions of glibc
 - Unsupported kernel interfaces



Why we might ask you to “use a newer version” of a scientific program, library etc.

- Security patching.
- Bug fixes, including those that affect the maths and therefore research outcomes.
- Updates may be required to support newer hardware or newer operating systems.
- Performance:
 - Sometimes there is an improvement if an update has optimised a section of code.
 - Sometimes there may be a performance regression but this might be for a good reason (see the first two comments above).

Check what hardware you are compiling your code on

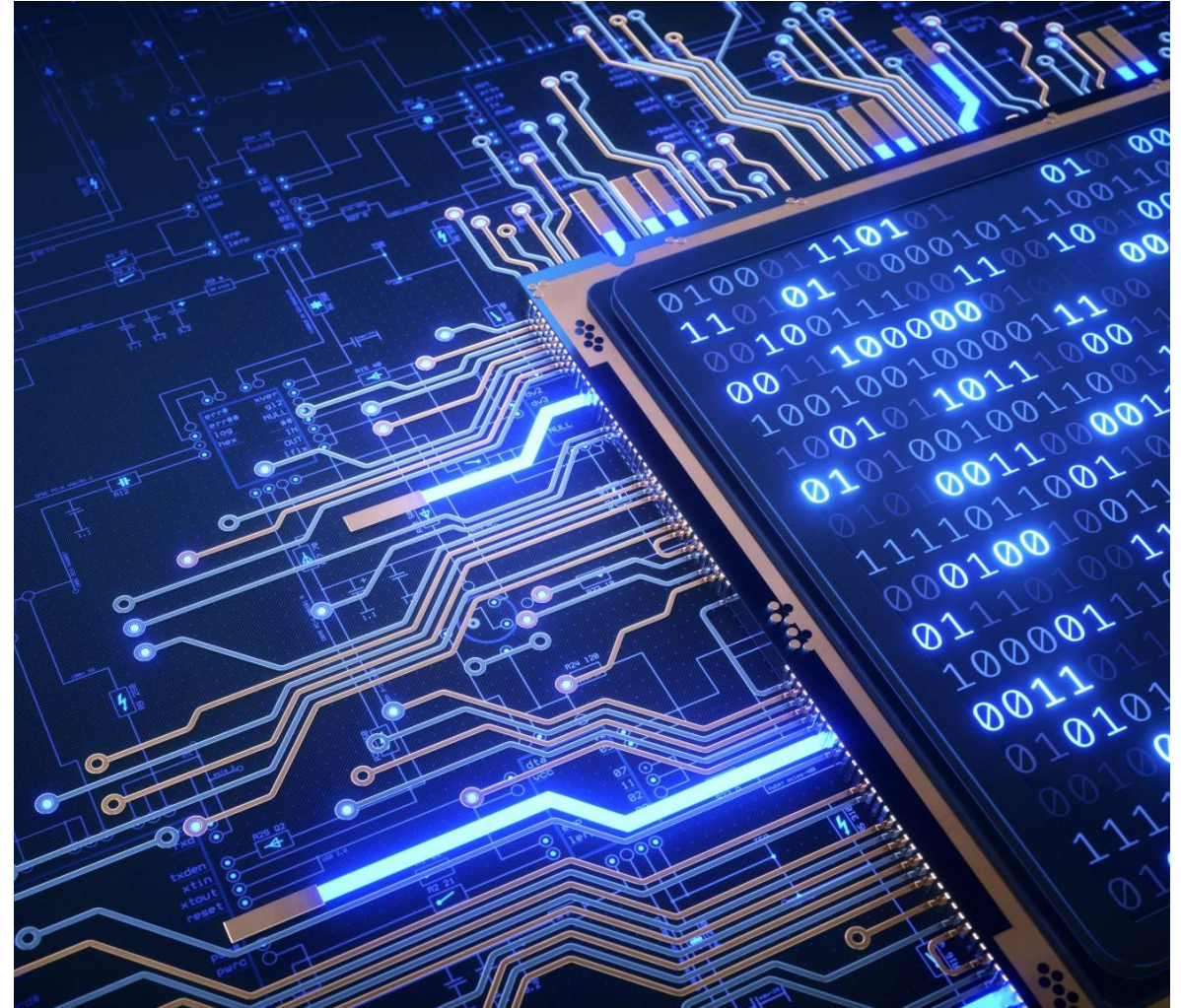
Login nodes often don't represent the hardware you are running your jobs on.

Many sites will also ask you not to compile on the login nodes as they are shared resources.

Consider building in an interactive job on the type of node you want to run on.

If there are different types of compute nodes available, you may find it beneficial to optimise for them (including having specific builds for each type):

1. CPU architecture e.g. ``march=native`` or ``-Xhost`` but be careful as these will optimise for every CPU feature the compiler can detect rather than a more recommended conservative list of features by specifying the architecture name.
2. CUDA Compute Capability
3. AMD GPU target architecture



Make it easier to build your application on different systems

Make sure it's not dependent on system-specific libraries, as this can make it difficult to build elsewhere: If your build instructions have `sudo apt install`, there's a good chance it will be difficult to build on an HPC system.

Have test jobs that you can use to:

- Check the validity of the code output (all the way through the code, not just in individual functions).
- Benchmark your code to find the optimal performance configuration.
- Every system behaves differently (different CPUs, networking, filesystem). You should make sure to run scaling tests to find the "layout" that gives you optimal performance rather than just running the largest job, as this is often not the fastest.

Scientific Software Build Tools

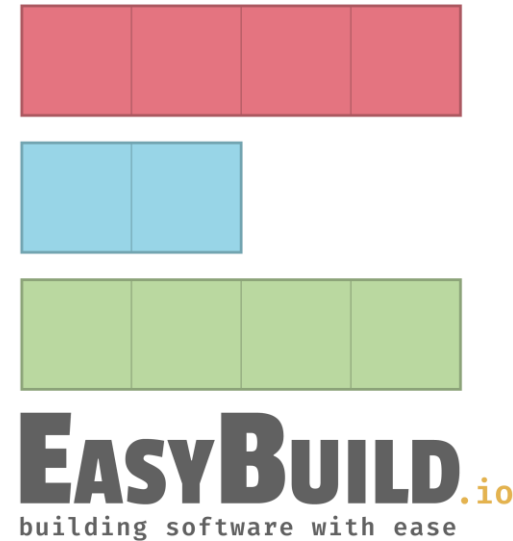
Many HPC sites use scientific software build tools to install software in a central location; users can then access those programs using “modules” (Environment Modules or Lmod).

Two commonly build tools are:

- EasyBuild (<https://easybuild.io/>)
- Spack (<https://spack.io/>)

EasyBuild and Spack are well suited for building traditional scientific applications, including OpenMP and MPI, offering convenient management of often complex library dependencies.

The rapid pace of AI library development (and build complexity) mean EasyBuild and Spack are struggling to keep up with release cycles and are therefore less useful for supporting such workload.



Consider committing and supporting pull requests describing how to build your software, into those build systems.

Or adopt design practices that simplify how others can build and your software (e.g., with tools like Makefiles).

The special case of python....

Python toolchains are becoming increasingly difficult for HPC teams to support centrally, due to their complex and sometimes conflicting library dependencies - particularly in AI workflows.



Python Environments

Manage your environments with tools such as pip, virtualenv, conda/mamba, uv but be aware that these tools can be very IO heavy (lots of small files with many read/write operations) and repeatedly running them may annoy your RIEs or your colleagues 😊

Anaconda ≠ conda!

- The Anaconda Distribution (including python) is governed by Anaconda Terms of Service / EULA. Check with your institution to make sure you are licensed to use it!
- Unless you need Anaconda packages, consider using conda with conda-forge repositories; miniforge is a good starting point for this.

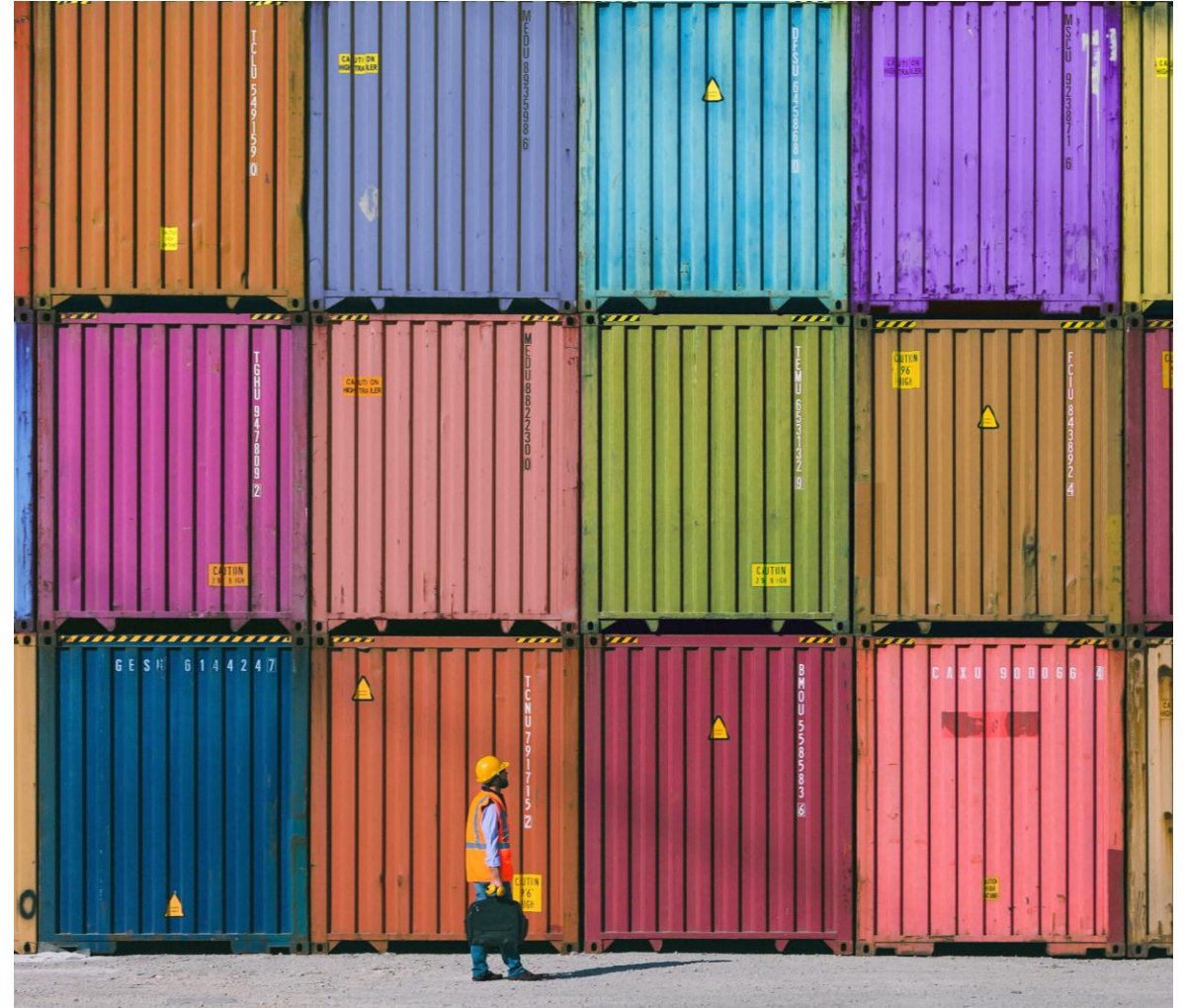
For AI frameworks, always check for the latest advice on their site (for example, pytorch no longer supports conda packages ([\[Announcement\] Deprecating PyTorch's official Anaconda channel · Issue #138506 · pytorch/pytorch](#)). If you install using conda, you will use community builds which may not be optimal.

What about containers?

They can help with complex builds, particularly those requiring multiple libraries and applications to be linked or those that require specific OS libraries.

They can sometimes make life easier for RIE's by bringing a ready-made workflow, but they're not always the best solution since:

- Your container is still locked to a specific OS version.
- That OS can get out of date, no longer supported, need patching (including security patching).
- That OS may not support the hardware you want to run on so even if your code runs, it may not run optimally.
- Container structures are not static, there are changes that can break compatibility.



Why don't HPC Centres (normally) support Docker

Docker was never really designed for a scientific computing workflow environment.

- Docker relies on a background service with root-level privileges, increased risk of privilege escalation by users interacting with Docker.
- Docker's traditional security model assumes a trusted local machine as compared with the HPC model of untrusted users sharing hardware safely.
- Docker assumes a different network and file system environment.
- Networking supports service-style communication for workloads (such as NAT) rather than multi-node high-performance communication.
- Root-owned overlay filesystems compared with shared parallel filesystems with HPC clusters.

Ultimately, it's about the Docker operational model. Instead, many HPC Centres support Apptainer that works well with HPC jobs schedulers:

- No root-daemon required and users can run containers without elevated permissions.
- Most, but not all, Docker containers can be converted to run as Apptainer containers.